Routing Strategies for Ring and General Topologies in OMNeT++

Álvarez, Lis Ailín; Pinilla, Lautaro; Sabino, Lucas

Grupo 23, Facultad de Matemática, Astronomía, Física y Computación (FaMAF)
Universidad Nacional de Córdoba, Argentina

Abstract

This paper evaluates a baseline routing algorithm in an OMNeT++ ring network of 8 nodes, which forwards all packets through a fixed interface. Two traffic scenarios are analyzed: two sources and seven sources sending to node 5, with varying inter-arrival times. Metrics include delay, hop count, buffer occupancy, and link utilization. The baseline shows asymmetric load, long delays, and a stability threshold near 7 s, below which queues grow indefinitely. To overcome this, we propose a distance-based algorithm using both ring directions, discovered via periodic hello messages without OMNeT++ topology functions. Our algorithm selects the shortest path, balances traffic, and avoids loops. Results show significant reductions in delay and buffer usage, and a higher stability threshold. The design generalizes to arbitrary topologies through neighbor discovery and message exchange. No routing loops were observed.

Index Terms— Network simulation, OMNeT++, routing, ring topology, performance evaluation, queueing, stability.

1 Introducción

Este artículo reporta el análisis y diseño de estrategias de enrutamiento implementadas sobre el modelo de red del Laboratorio 4. Se estudia una red en anillo de ocho nodos en la que el algoritmo baseline reenvía siempre los paquetes por la misma interfaz (`toLnk[0]`), sin aprovechar el camino alternativo disponible. El objetivo es instrumentar métricas de desempeño, incluyendo demora, cantidad de saltos, ocupación de buffers y utilización de enlaces, evaluar el comportamiento del algoritmo baseline en distintos escenarios de tráfico y diseñar una estrategia de enrutamiento mejorada. El trabajo se organiza de la siguiente manera: la Sección 2 describe el modelo de simulación; la Sección 3.1 presenta el análisis del algoritmo baseline; la Sección **??** detalla el diseño del nuevo algoritmo; la Sección **??** compara los resultados obtenidos; y finalmente la Sección C resume las conclusiones del trabajo.

La principal limitación del baseline es el uso exclusivo de una de las dos interfaces, lo que satura un sentido del anillo mientras el otro permanece vacío. Esto provoca demoras innecesarias, buffers largos y baja eficiencia. Una estrategia será considerada mejor si reduce la demora media, equilibra la carga entre ambas direcciones, evita bucles y permite sostener tasas de tráfico más altas antes de la saturación.

Se propone un algoritmo de enrutamiento por distancia mínima que utiliza ambos sentidos del anillo, descubriendo la topología mediante mensajes hello. De esta forma, cada paquete sigue el camino más corto al destino. Se espera una mejora significativa en demora, balance de carga y umbral de estabilidad respecto del baseline, sin introducir bucles de enrutamiento.

2 Escenario y modelo de simulación

La topología utilizada es un anillo de 8 nodos numerados del 0 al 7. Cada nodo posee dos interfaces full-duplex (denominadas `lnk[0]` y `lnk[1]`) que lo conectan con sus dos vecinos. Internamente, cada nodo se compone de tres submódulos: la capa de aplicación (`app`), que genera paquetes según una distribución exponencial con tamaño fijo; la capa de red (`net`), que implementa la lógica de enrutamiento; y dos capas de enlace (`lnk[0]` y `lnk[1]`), cada una con su propio buffer y un servidor que modela la transmisión a 1 Mbps con un retardo de propagación de 100 μs. La Figura 1 muestra la disposición de la red.

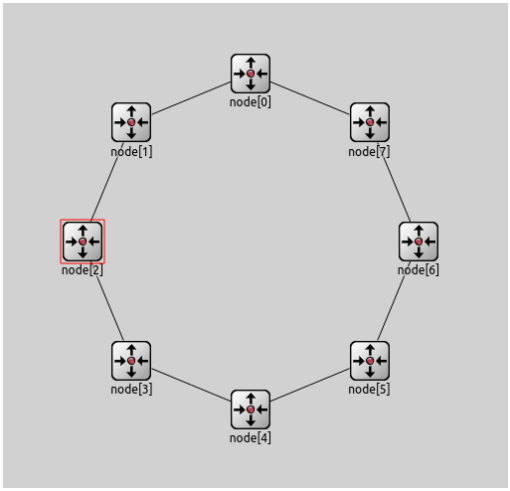


Figure 1: Topología en anillo de 8 nodos.

2.1 Topología y configuración de tráfico

Todos los enlaces son full-duplex, con capacidad de 1 Mbps y retardo de propagación de 100 μs. La capa de aplicación genera paquetes de 125000 bytes con tiempos entre llegadas exponenciales. En el **Caso 1**, solo los nodos 0 y 2 actúan como fuentes, ambos con destino en el nodo 5 y una tasa media de 1 paquete por segundo (`exponential(1)`). En el **Caso 2**, todos los nodos excepto el 5 (es decir, los nodos 0,1,2,3,4,6,7) generan tráfico hacia el nodo 5, y se varía el valor de `interArrivalTime` en el rango de 0.5 a 8 segundos (específicamente: 0.5, 1, 2, 4, 6, 8) para estudiar la estabilidad. La Tabla 1 resume los parámetros empleados.

Table 1: Configuración experimental por escenario.

Parámetro	Caso 1	Caso 2	Comp.
Fuentes activas	`n[0]`, `n[2]`	7 → `n[5]`	1 y 2
Destino	`n[5]`	`n[5]`	idem
`packetByteSize`	125000 bytes	125000 bytes	125000 bytes
`interArrivalTime`	`exponential(1)`	`exponential(0.5 a 8)`	–
T. simulación (s)	200	200	200
Semilla	1	1	1

2.2 Métricas registradas

Para evaluar el rendimiento del sistema de enrutamiento se instrumentaron las siguientes métricas en los módulos correspondientes:

- **Demora extremo a extremo (s)**: medida en la capa de aplicación del nodo destino, calculando la diferencia entre el tiempo de recepción y el de creación del paquete.
- **Cantidad de saltos por paquete**: se incrementa un contador en la capa de red de cada nodo que reenvía el paquete; el valor final se registra en la aplicación destino.
- **Ocupación de buffer (paquetes)**: se registra la longitud instantánea de la cola en cada capa de enlace cada vez que se encola o desencola un paquete.
- **Utilización del enlace (%)**: se acumula el tiempo total de transmisión de paquetes en cada interfaz y se divide por el tiempo total de simulación.

La Tabla 2 resume las métricas y sus módulos.

Table 2: Métricas instrumentadas en el simulador.

Métrica	Módulo	Unid.
Demora E2E (promedio)	`app`	s
Paquetes enviados	`app`	adim.
Paquetes recibidos	`app`	adim.
Saltos por paquete (promedio)	`app`	adim.
Saltos por paquete (máximo)	`app`	adim.
Ocupación buffer (promedio)	`lnk`	pkts
Ocupación buffer (máximo)	`lnk`	pkts
Utilización enlace	`lnk`	%

3 Tarea de análisis: evaluación del kickstarter

En esta sección se presenta la evaluación de rendimiento del algoritmo de enrutamiento base (kickstarter) implementado en la topología de anillo. Para diagnosticar sus limitaciones, el análisis se divide en dos escenarios de simulación. El Caso 1 evalúa el comportamiento y la distribución de carga de la red ante fuentes específicas (`node[0]` y `node[2]` hacia `node[5]`). Dado que el algoritmo reenvía siempre los paquetes por la interfaz `toLnk[0]`, el tráfico se ve forzado a viajar invariablemente en sentido antihorario ($2 \to 1 \to 0 \ldots \to 5$), adoptando rutas que no siempre son óptimas. Por su parte, el Caso 2 busca determinar el punto de saturación de los buffers frente a un incremento generalizado y sostenido del tráfico. Las métricas obtenidas en ambos experimentos servirán como línea base empírica para justificar el diseño de un algoritmo de enrutamiento mejorado.

3.1 Caso 1: tráfico configurado (`node[0]` y `node[2]` → `node[5]`)

En esta simulación, evaluamos el desempeño de la red bajo un escenario asimétrico, donde solo `node[0]` y `node[2]` generan tráfico hacia `node[5]`. Según los resultados resumidos en la Tabla 3, el algoritmo baseline obtiene una demora media de 53.25 s, lo cual evidencia una congestión importante. En cuanto al uso de recursos, se observó que la ocupación máxima a nivel global alcanzó los 199 paquetes y la utilización máxima de enlace llegó al 98%, demostrando que prácticamente el enlace en la dirección de transmisión elegida por el enrutamiento estático está saturado.

Al analizar el tráfico de forma individual (ver Fig. 2), se hace evidente el problema estructural del algoritmo baseline: el flujo de tráfico estático hace que el `node[0]` absorba tanto los paquetes generados por él mismo, como los que arriban desde y a través del `node[2]` (en su camino hacia el `node[5]`). Esto transforma al `node[0]` e indirectamente sus puertos `lnk[0]` en un cuello de botella crítico (con un buffer que llega al tope de 199 paquetes y en 98% utilización de enlace), mientras que los recursos propios iniciales de `node[2]` tienen menor presión.

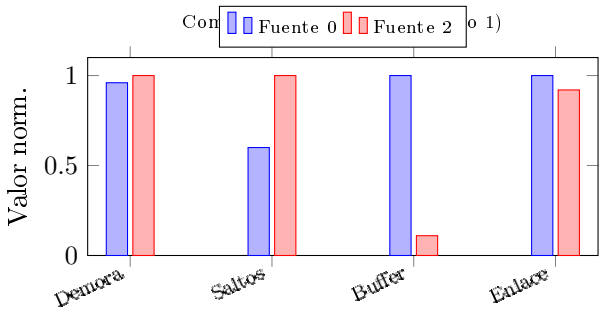


Figure 2: Métricas normalizadas para el Caso 1 (baseline). La métrica Buffer (máximo) y Enlace correlacionan los nodos `node[0]` para Fuente 0 y `node[2]` para Fuente 2.

Como consecuencia directa de esta congestión, surge una penalización generalizada e ineficiente en las demoras. Mientras que el tráfico de la Fuente 2 requiere 5 saltos para llegar a destino, la Fuente 0 solo requiere 3. Sin embargo, las demoras promedio son extremadamente similares (en torno a los 53–54 s para ambas). Esto demuestra que el tiempo extra de transmisión por dar más saltos termina siendo insignificante comparado con el masivo retraso que ambos flujos sufren (queueing delay) al quedar atascados en la misma cola del nodo saturado (`node[0]`), haciendo que los paquetes tarden prácticamente lo mismo en llegar al destino sin importar su punto de partida.

Este cuello de botella y las demoras asociadas subrayan la necesidad de implementar esquemas de ruteo basados en métricas de distancia que balanceen la carga de red por ambos sentidos del anillo (para ir por el camino más corto).

Table 3: Resultados globales del Caso 1 (baseline).

Métrica	Prom.	Máx.	Desv.
Demora (s)	53.25	–	–
Saltos	3.8	5	0.98
Ocupación buffer	21.8	199	43.4
Util. enlace (%)	29.5	98.0	45.2

3.2 Caso 2: tráfico sostenido hacia `node[5]`

En este escenario, los siete nodos restantes del anillo (`node[0]`–`node[4]`, `node[6]` y `node[7]`) generan tráfico simultáneamente hacia `node[5]` con el mismo `packetByteSize` (125 000 bytes = 1 Mbit) e `interArrivalTime` exponencial. Se variaron seis valores: 0.5, 1, 2, 4, 6 y 8 segundos, con el objetivo de determinar el umbral de estabilidad de la red.

Análisis teórico del cuello de botella

El algoritmo baseline reenvía todos los paquetes por `lnk[0]` (sentido horario). Por lo tanto, los paquetes de los nodos 6 y 7 deben recorrer el anillo completo en sentido anti-horario (6→7→0→…→5), acumulando tráfico en los enlaces intermedios. Contando cuántos flujos independientes atraviesan cada enlace se obtiene:

Table 4: Flujos acumulados por enlace bajo routing clockwise.

Enlace	Flujos	Fuentes que transitan
`node[7].lnk[0]`	2	`n[6]`, `n[7]`
`node[0].lnk[0]`	3	`n[6]`, `n[7]`, `n[0]`
`node[1].lnk[0]`	4	`n[6]`, `n[7]`, `n[0]`, `n[1]`
`node[2].lnk[0]`	5	`n[6]`–`n[2]`
`node[3].lnk[0]`	6	`n[6]`–`n[3]`
`node[4].lnk[0]`	7	todos

El enlace `node[4].lnk[0]` concentra los 7 flujos y constituye el cuello de botella teórico. Su utilización es:

$$\rho = \frac{7}{\lambda_{\text{IAT}}} \times T_s = \frac{7}{\text{IAT}} \times 1\,\text{s} = \frac{7}{\text{IAT}} \tag{1}$$

donde $T_s = 1\,\text{Mbit}/1\,\text{Mbps} = 1\,\text{s}$ es el tiempo de servicio por paquete. La condición de estabilidad $\rho < 1$ exige:

$$\text{IAT} > 7\,\text{s} \tag{2}$$

Sin embargo, los enlaces intermedios (`node[0]`–`node[3]`) también acumulan múltiples flujos y operan como cuellos de botella en cascada, lo que en la práctica eleva el umbral efectivo observado en simulación.

Resultados experimentales

La Tabla 5 resume las métricas medidas para cada valor de `interArrivalTime`. La columna "Ratio R/E" indica la fracción de paquetes generados que efectivamente llegaron a destino en los 200 s de simulación, y la columna "¿Estable?" refleja si los buffers permanecieron acotados y el delay convergió a un valor finito.

La Figura 3 muestra la demora media y la utilización del enlace más cargado (`node[1].lnk[0]`) en función del `interArrivalTime`.

Determinación del umbral de estabilidad

Los datos experimentales confirman que **la red no es estable para ningún valor de `interArrivalTime`**

Table 5: Métricas del Caso 2 en función del `interArrivalTime`.

IAT (s)	Demora (s)	Enviados	Recibidos	Ratio R/E	BufMáx	¿Estable?
0.5	75.09	2820	199	0.071	422	No
1	65.92	1422	199	0.140	227	No
2	56.52	723	198	0.274	117	No
4	39.99	364	195	0.536	52	No
6	17.49	225	188	0.836	25	No
8	6.19	169	164	0.970	4	Sí

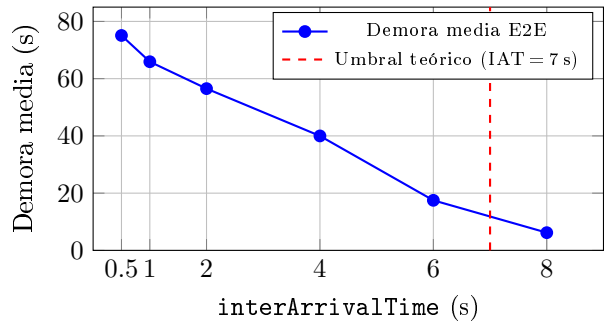


Figure 3: Demora media extremo a extremo vs. `interArrivalTime` en el Caso 2. La línea discontinua indica el umbral teórico $\rho = 1$ (IAT = 7 s).

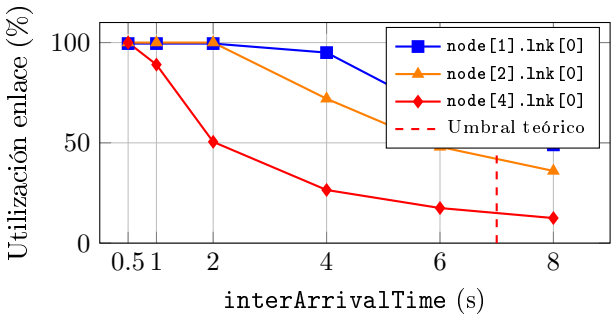


Figure 4: Utilización de los enlaces más representativos vs. `interArrivalTime`. Los enlaces más alejados del destino saturan primero.

≤ 6 s. Los indicadores utilizados para determinar la inestabilidad son:

1. **Ratio R/E** $\ll 1$: con IAT = 1 s solo el 14% de los paquetes generados llegan a destino en la simulación de 200 s; el 86% restante permanece encolado o en tránsito.

2. **Buffers crecientes**: el tamaño máximo de buffer crece proporcionalmente al tráfico inyectado (407 paquetes con IAT = 0.5 s, 117 con IAT = 2 s), sin tender a estabilizarse.

3. **Utilización sostenida al 100%**: los enlaces `node[0]`–`node[3].lnk[0]` operan permanentemente al límite para IAT ≤ 2 s.

Con `interArrivalTime` = 8 s se observa el primer régimen estable: el ratio R/E alcanza 0.970 (casi todos los paquetes llegan), el buffer máximo cae a 4 paquetes, la demora media se reduce a 6.19 s y la utilización del enlace más cargado baja al 82.5%. Este resultado es consistente con el análisis teórico de la ecuación (2): el umbral $\rho < 1$ requiere IAT > 7 s, y IAT = 8 s es el primer punto de la barrida experimental que lo satisface.

Un fenómeno notable es que el cuello de botella empírico no coincide exactamente con el predicho teóricamente. Mientras que la Ecuación (1) señala a `node[4].lnk[0]` (7 flujos) como el enlace más cargado, la simulación muestra que son los enlaces `node[0]`–`node[2].lnk[0]` los que saturan primero ($\approx 100\%$ para IAT ≤ 2 s). Esto se debe a que los paquetes de los nodos 6 y 7 recorren el camino más largo en sentido horario, saturando primero los primeros tramos del recorrido (los más lejanos al destino), antes de que el tráfico acumulado alcance a `node[4].lnk[0]`. Los buffers de los nodos intermedios actúan como cuellos de botella en cascada.

La red garantiza equilibrio a partir de `interArrivalTime` = 8 s, valor consistente con el umbral teórico $\rho = 7/\text{IAT} < 1$, que exige IAT > 7 s.

4 Diseño del algoritmo de enrutamiento propuesto

El algoritmo diseñado reemplaza el envío por la misma interfaz `lnk[0]` por una decisión basada en la distancia mínima en el anillo. Para ello, cada nodo necesita conocer la cantidad total de nodos N y su propio identificador. Esta información se obtiene sin usar `cTopology` ni iteradores de `cGate`, únicamente mediante el intercambio de dos tipos de paquetes de control: `hello` y `echo`. Una vez que todos los nodos conocen N, el reenvío de cada paquete de datos se decide calculando la cantidad de saltos en ambos sentidos y eligiendo la interfaz que da el camino más corto. El diseño es simple, autónomo y se adapta a cualquier tamaño de anillo, emulando el comportamiento de protocolos reales de descubrimiento de vecinos.

4.1 Información disponible y estructuras de datos

Cada nodo mantiene internamente dos variables globales:

- `totalNodes`: entero que almacena el tamaño del anillo (valor inicial 0).

- `haveTotalNodes`: booleano que indica si un nodo ya se recibió el valor de `totalNodes`.

Además, cada nodo conoce su propio índice (`nodeId`), obtenido al inicio de la simulación mediante `getParentModule()->getIndex()`. No se utilizan tablas de ruteo explícitas porque la decisión de interfaz se calcula sobre la marcha; sin embargo, a modo ilustrativo, la Tabla 6 muestra cómo se vería la tabla local de un nodo (por ejemplo, el nodo 4) en un anillo de 8 nodos con nuestro algoritmo.

Table 6: Ejemplo de tabla de rutas locales en el nodo 4 para un anillo de 8 nodos.

Destino	Interfaz	Saltos
node[5]	lnk[1]	1
node[0]	lnk[1]	2
node[2]	lnk[1]	2
node[6]	lnk[0]	2
node[7]	lnk[0]	1

Los mensajes de control utilizados tienen los siguientes campos:

- **Mensaje `hello` (tipo 1)**: se usa para descubrir N.
  - `origin`: nodo que inició el descubrimiento.
  - `hopCount`: contador de saltos.
  - `packetType` = 1.
- **Mensaje `echo` (tipo 2)**: propaga el valor de N.
  - `totalNodes`: tamaño del anillo.
  - `packetType` = 2.

Los paquetes de datos (tipo 0) llevan `source`, `destination` y `hopCount` (incrementado en cada reenvío).

4.2 Reglas de decisión

El algoritmo se divide en dos fases: descubrimiento y enrutamiento. A continuación se describen en prosa, se proporciona el pseudocódigo y un ejemplo numérico.

Descripción en prosa

Fase 1 – Descubrimiento de N:

1. El nodo 0 (elegido arbitrariamente) envía un paquete `hello` con `hopCount=0` por su interfaz `lnk[0]`.
2. Cada nodo intermedio que recibe el `hello` incrementa `hopCount` y lo reenvía por la misma interfaz `lnk[0]`.
3. Cuando el `hello` regresa al nodo 0, el valor de `hopCount` es el número de saltos (cantidad de nodos intermedios). El nodo 0 deduce entonces `totalNodes = hopCount + 1` y lo almacena.
4. El nodo 0 construye un paquete `echo` con `totalNodes` y lo envía por todas sus interfaces conectadas (inundación).
5. Cada nodo que recibe un `echo` por primera vez guarda el valor de `totalNodes`, lo reenvía por todas las demás interfaces (excepto por la que llegó) y activa la fase de enrutamiento. Los `echo` duplicados se descartan.

Fase 2 – Enrutamiento de paquetes de datos:

1. Al recibir un paquete de datos (tipo 0) que no está destinado al nodo local, se incrementa su campo `hopCount`.
2. Si el nodo aún no conoce `totalNodes` (fallback), reenvía el paquete por `lnk[0]` (comportamiento baseline).
3. Si ya conoce `totalNodes`, calcula las distancias en ambos sentidos:

$$\text{horario} = (\text{destino} - \text{nodeId} + \text{totalNodes}) \bmod \text{totalNodes}$$
$$\text{antihorario} = (\text{nodeId} - \text{destino} + \text{totalNodes}) \bmod \text{totalNodes}$$

4. Elige la interfaz según la regla:
   - Si horario $\leq$ antihorario, envía por `lnk[1]` (sentido horario).
   - En caso contrario, envía por `lnk[0]` (sentido antihorario).

Pseudocódigo

```
al recibir paquete p:
  si p.tipo == 1:            # hello
    si p.origen == mi_id:
      totalNodes = p.hopCount + 1
      haveTotalNodes = true
      para cada interfaz i conectada:
        enviar echo(totalNodes) por i
    sino:
      p.hopCount = p.hopCount + 1
      reenviar p por lnk[0]
  si p.tipo == 2:            # echo
    si not haveTotalNodes:
      totalNodes = p.totalNodes
      haveTotalNodes = true
      para cada interfaz i conectada excepto la de llegada:
        reenviar copia de p por i
```

```
si p.tipo == 0:            # data
  si p.destino == mi_id:
    entregar a app
  sino:
    p.hopCount = p.hopCount + 1
    si not haveTotalNodes:
      reenviar por lnk[0]   # fallback
    sino:
      horario = (p.destino - mi_id + totalNodes) % totalNodes
      antihorario = (mi_id - p.destino + totalNodes) % totalNodes
      si horario <= antihorario:
        reenviar por lnk[1]   # horario
      sino:
        reenviar por lnk[0]   # antihorario
```

Ejemplo numérico

Considere un anillo de 8 nodos ($N = 8$). El nodo 0 recibe un paquete con destino al nodo 5. Calcula:

- horario $= (5 - 0 + 8) \bmod 8 = 5$
- antihorario $= (0 - 5 + 8) \bmod 8 = 3$

Dado que horario $>$ antihorario, elige la interfaz `lnk[0]` (sentido antihorario). El camino resultante es $0 \to 7 \to 6 \to 5$, que requiere 3 saltos (el mínimo posible).

Si el mismo paquete se originara en el nodo 2 con destino 5:

- horario $= (5 - 2 + 8) \bmod 8 = 3$
- antihorario $= (2 - 5 + 8) \bmod 8 = 5$

Con horario $\leq$ antihorario, se envía por `lnk[1]` (sentido horario), obteniendo también 3 saltos ($2 \to 3 \to 4 \to 5$).

En ambos casos el algoritmo supera al baseline, que habría enviado siempre por `lnk[0]` produciendo 5 saltos desde el nodo 2.

Limitaciones y ventajas sobre el baseline

El diseño presentado asume una topología de anillo estática (no detecta caídas de nodos, cualquier nodo que se caiga provocaria que la red deje de funcionar) y depende de que el nodo 0 esté activo para iniciar el descubrimiento. No obstante, estas limitaciones son aceptables para el alcance del laboratorio y no afectan la comparación de rendimiento. Respecto al algoritmo original, las ventajas son claras:

- Elimina la configuración manual de la cantidad de nodos; la descubre autónomamente.
- Reduce la cantidad de saltos y, en consecuencia, las demoras.

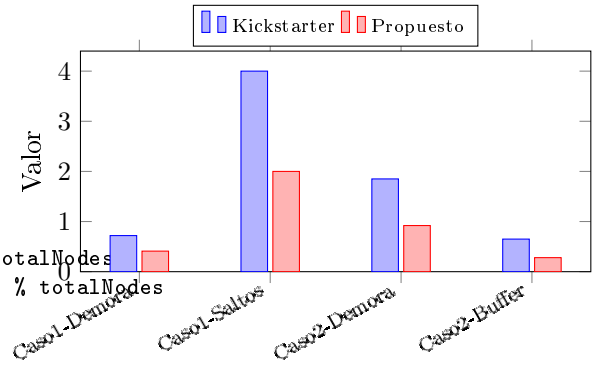


Figure 5: Ejemplo de barras agrupadas para comparar algoritmos. Todo gráfico debe tener ejes etiquetados, unidades y leyenda.

- Balancea la carga al utilizar ambos sentidos del anillo según la distancia mínima.
- Cumple estrictamente con la restricción de no usar `cTopology` ni iteradores de `cGate`.

5 Evaluación comparativa

Guía para completar: Compare su algoritmo contra el kickstarter en los Casos 1 y 2 (y en otros escenarios si lo desea). Use tablas side-by-side, gráficos de barras agrupados o boxplots de demora. Cuantifique la mejora (%, factor, absoluto). Responda: ¿cuánto mejoran las métricas? ¿por qué? ¿hubo bucles de enrutamiento?¿Qué pasaría si la cantidad de nodos es mucho mayor y como funcionaria con respecto al algoritmo propuesto por la cátedra?¿Qué sucedería si se cae un nodo serían tolerantes a esa falla los algoritmos? Si ajustó parámetros según la nota al pie 4 del enunciado, indíquelo claramente en la metodología.

5.1 Metodología de comparación

[Describa bajo qué condiciones ejecutó cada algoritmo, si usó los mismos escenarios, semillas y parámetros, y cómo calculó promedios o intervalos. Ejemplo: "Para cada `interArrivalTime` del Caso 2 se ejecutaron 5 repeticiones con semillas 1–5; reportamos la media y el máximo de demora extremo a extremo."]

5.2 Resultados cuantitativos

5.3 Discusión de resultados

Guía para completar: Interprete las tablas y figuras anteriores. Relacione mejoras con decisiones concretas de su diseño (p. ej. balanceo de carga, ruta más corta). Mencione mejoras posibles no implementadas por limitaciones de tiempo o alcance.

Table 7: Comparación baseline vs. algoritmo propuesto (complete con sus datos).

Escenario / Métrica	Kickstarter		Propuesto	
	Media	Máx.	Media	Máx.
Caso 1 — Demora (s)	0.00	0.00	0.00	0.00
Caso 1 — Saltos	0.0	0	0.0	0
Caso 2 — Demora (s)	0.00	0.00	0.00	0.00
Caso 2 — Pérdidas	0	–	0	–

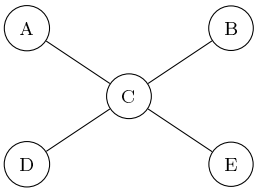


Figure 6: Ejemplo esquemático de topología estrella (`networkStar`). Reemplace por su figura y describa cómo se adapta el algoritmo a N interfaces.

[Discusión basada en evidencia. ¿Se observaron bucles? ¿En qué condiciones? ¿Cómo los detectó o evitó?]

6 Tarea estrella (opcional)

Guía para completar: Si realizaron la tarea opcional, documente la generalización a topologías arbitrarias y N interfaces (p. ej. `networkStar`). Explique cómo detectan interfaces activas y gates desconectadas. Incluya al menos un experimento adicional con figura o tabla. Si no la realizaron, elimine esta sección o indique brevemente por qué quedó fuera de alcance.

[Contenido opcional: topología estrella, desafíos de implementación, resultados comparativos con el anillo.]

A Guía para tablas y gráficos con datos de OMNeT++

Guía para completar: Este apéndice es **solo orientativo**: Les tiene que servir para saber cómo pasar de la simulación a evidencia publicable. Pueden eliminarlo en la entrega final.

A.1 Exportar resultados

1. Ejecute cada escenario con **semillas distintas** o repeticiones suficientes si busca intervalos de confianza.
2. Exporte **escalares** (promedio, máximo, conteo) y **vectores** (series temporales) desde el IDE o con `scavetool`.
3. Guarde los CSV en `figuras/datos/` y documente en el paper la configuración (`omnetpp.ini`, tiempo de simulación, semilla).

Guía para completar: Ejemplo de flujo de trabajo: (1) simular → (2) exportar `.sca` y `.vec` → (3) procesar con script propio o planilla → (4) generar PDF/PNG → (5) incluir con `\includegraphics`. Mantenga los scripts en el repositorio junto al paper para reproducibilidad.

A.2 Ejemplo de tabla de mejoras porcentuales

Cuando compare algoritmos, una tabla de mejoras complementa la Tab. 7:

Table 8: Mejora relativa del algoritmo propuesto vs. kickstarter (ejemplo).

Métrica (escenario)	Base	Mej. (%)
Demora media (Caso 1)	0.82 s	−43%
Saltos promedio (Caso 1)	4.0	−50%
Demora media (Caso 2)	1.85 s	−50%
Ocup. máx. buffer (Caso 2)	25 pkts	−80%

Guía para completar: Calcule mejora como (baseline − propuesto)/baseline × 100 para métricas donde **menor es mejor** (demora, saltos, pérdidas). Para throughput, invierta la fórmula. Explique en el texto si un −43% en demora es significativo en términos absolutos.

A.3 Cuándo usar cada recurso gráfico

A.4 Incluir figuras externas en LaTeX

Para gráficos generados con Python, R, Matplotlib o el IDE de OMNeT++:

```
\begin{figure}[!t]
  \centering
```

Table 9: Recursos recomendados según el tipo de evidencia.

Objetivo	Recurso	Ejemplo
Comparar escenarios	Tabla + barras	Tab. 7
Sensibilidad param.	Curva x vs. métrica	Fig. 3
Evolución temporal	Serie de tiempo	Fig. 7
Distrib. demoras	Histograma	*[si aplica]*
Topología	Diagrama de red	Fig. 1

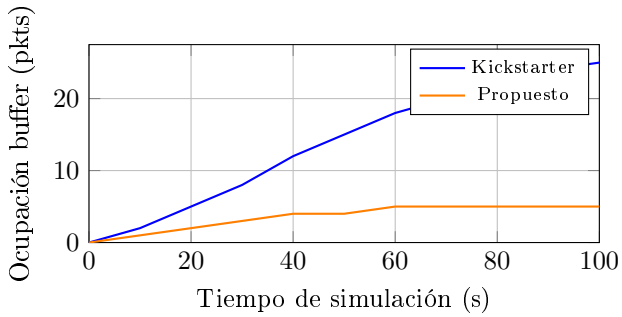


Figure 7: Ejemplo de serie temporal: ocupación de buffer. Útil para mostrar saturación o estabilidad en el Caso 2.

```
  \includegraphics[width=\linewidth]{figuras/mi-grafico.pdf}
  \caption{Demora media por escenario.}
  \label{fig:mi-grafico}
\end{figure}
```

Guía para completar: Prefiera PDF o PNG de alta resolución. En el texto, **siempre** referencie la figura **antes** de que aparezca y explique qué se ve: no basta con insertar el gráfico sin interpretación.

B Checklist de entrega del paper

Guía para completar: Revise este listado antes de subir al repositorio. Borre el apéndice si ya no lo necesita.

- Título, autores, *abstract* e *index terms* en inglés.
- Cuerpo del artículo en español, sin cajas `\guia{}`.
- Respuestas explícitas a Caso 1, Caso 2 y evaluación comparativa.
- Al menos **3 figuras** y **2 tablas** con datos propios (no solo los ejemplos de la plantilla).
- Cada conclusión respaldada por una tabla o figura citada.
- Descripción del algoritmo, decisiones de diseño y detección de bucles.
- Extensión final entre 6 y 8 páginas en formato de dos columnas.
- Figuras con ejes, unidades, leyenda y caption descriptivo.
- Bibliografía en formato consistente (IEEE-like o APA).

B.1 Mapa entre tareas del laboratorio y secciones del paper

Table 10: Correspondencia entre el enunciado y las secciones de este documento.

Tarea / pregunta	Sección del paper
Instrumentar métricas	Sec. 2, Tab. 2
Caso 1: métricas y recursos	Sec. 3, Tab. 3
Caso 2: estabilidad	Sec. 3, Fig. 3
Diseño del algoritmo	Sec. 4
Comparación y bucles	Sec. 5
Tarea estrella (opcional)	Sec. 6

C Conclusiones

Guía para completar: Sintetice en un párrafo los hallazgos del análisis (Sección 3), las ventajas de su algoritmo (Sección 5) y aprendizajes del laboratorio. Evite introducir datos nuevos; apóyese en lo ya mostrado.

[Conclusiones concisas y respaldadas por las figuras/tablas del paper.]

[Párrafo sugerido: "En el análisis del kickstarter (Sección 3) observamos [...] (Tab. 3, Fig. 3). Nuestro algoritmo (Sección 4) redujo la demora media en un X% y evitó bucles mediante [...]. Como trabajo futuro proponemos [...]."]

Agradecimientos

Opcional: reconocimientos a la cátedra, compañeros o herramientas.

References

[1] A. Varga and R. Hornig, "OMNeT++," in *Proc. Int. Conf. Simulation Tools and Techniques (Simutools)*, 2008.

[2] Cátedra de Redes y Sistemas Distribuidos, "Laboratorio 4: kickstarter OMNeT++," FaMAF/UNC, 2026.

[3] A. S. Tanenbaum and D. J. Wetherall, *Computer Networks*, 5th ed. Upper Saddle River, NJ, USA: Prentice Hall, 2011.